

Bases de Datos

Clase 25: Búsqueda de Texto — PostgreSQL, BM25 y Embeddings

Miércoles 3 de junio

Recordatorio de la clase anterior

- **Índice invertido**: término $\rightarrow$ posting list de documentos.
- Pipeline al indexar: **tokenización** $\rightarrow$ **normalización** $\rightarrow$ **stop words** $\rightarrow$ **stemming**.
- **Búsqueda booleana** vía intersección/unión de posting lists.
- **Ranking** con **TF-IDF**: $TF(t, d) \cdot \log(N / df(t))$.

Hoy: todo esto en PostgreSQL real, **BM25** (el ranking de producción), y una mirada a búsqueda vectorial.

PostgreSQL: tsvector y tsquery

to_tsvector: el pipeline en SQL

Convierte un texto en un valor **tsvector** — la representación pre-procesada, lista para indexar.

```
SELECT to_tsvector('spanish', 'Las bases de datos son fundamentales');
```

```
'base':2 'dato':4 'fundamental':6
```

PostgreSQL automáticamente:

- Tokenizó el texto con el diccionario de español.
- Quitó stop words (**las** , **de** , **son**).
- Aplicó stemming (**bases** → **base** , **datos** → **dato**).
- Guardó las posiciones de cada lexema.

→ Todo el pipeline de la clase pasada, en **una función**.

tsvector por dentro

```
SELECT to_tsvector('spanish', 'El gato come pescado. El pescado es del gato.');
```

```
'come':3 'gat':2,9 'pescad':4,6
```

- Cada lexema aparece **una vez**, con la lista de **posiciones**.
- `gat` (stem de `gato`) en posiciones 2 y 9 — útil para frases y para calcular TF.
- `el`, `es`, `del` eliminados como stop words.

to_tsquery y operador @@

La **consulta** pasa por el mismo pipeline; @@ hace el match.

```
SELECT to_tsquery('spanish', 'bases & datos');           -- 'base' & 'dato'
SELECT to_tsvector('spanish', 'Las bases de datos son fundamentales')
       @@ to_tsquery('spanish', 'bases & datos');         -- TRUE
```

Operador en tsquery	Significado
&	AND
	OR
!	NOT
<->	seguido (frase)

Mismo pipeline al indexar y al consultar — sin eso, **nunca habría match**.

Ejemplo: tabla de artículos

```
CREATE TABLE articulos (  
    id          SERIAL PRIMARY KEY,  
    titulo     TEXT,  
    cuerpo     TEXT  
);  
  
SELECT id, titulo  
FROM articulos  
WHERE to_tsvector('spanish', titulo || ' ' || cuerpo)  
@@ to_tsquery('spanish', 'bases & datos');
```

Funciona — pero sin índice, PostgreSQL recorre **toda la tabla**, recomputando el `tsvector` por cada fila.

→ Necesitamos el **índice invertido de la clase pasada**, en PostgreSQL.

Crear el índice invertido

```
CREATE INDEX articulos_tsv_idx
ON articulos
USING GIN (to_tsvector('spanish', titulo || ' ' || cuerpo));
```

- **USING GIN**: el tipo de índice que PostgreSQL provee para **tsvector**.
- Internamente: **diccionario de lexemas** → **posting list** de filas. Exactamente lo de la clase pasada.
- Pagamos en inserción (cada documento toca varias posting lists), ganamos órdenes de magnitud en lectura.

Detrás de **USING GIN** hay un **índice invertido**. Los detalles de cómo se almacena en disco... en la documentación.


```
EXPLAIN ANALYZE
SELECT id, titulo FROM articulos
WHERE to_tsvector('spanish', titulo || ' ' || cuerpo) @@ to_tsquery('spanish', 'bases & datos');
```

Sin índice:

```
Seq Scan on articulos
  Filter: (to_tsvector(...) @@ ...)
```

Con índice:

```
Bitmap Heap Scan on articulos
  -> Bitmap Index Scan on articulos_tsv_idx
```

ts_rank — y por qué no basta

```
SELECT id, titulo, ts_rank(tsv, query) AS rank
FROM   artículos, to_tsquery('spanish', 'bases & datos') AS query
WHERE  tsv @@ query
ORDER BY rank DESC LIMIT 10;
```

- Considera **frecuencia** y **pesos** por sección (**A** título, **B** cuerpo, ...).
- **Pero no es TF-IDF completo**: no usa el **df** global del corpus → **sin IDF real**.

Para ranking en serio, necesitamos algo mejor. La industria no usa TF-IDF puro ni **ts_rank** — usa **BM25**.

BM25 — el ranking real

TF-IDF tiene dos problemas

1. TF crece sin freno.

Un documento que menciona "gato" 100 veces no es mucho más relevante que uno que lo menciona 10 veces. La utilidad de cada nueva aparición **satura**.

2. Los documentos largos ganan por defecto.

Un libro entero acumula más TF que un párrafo bien dirigido al tema. Sin normalización por largo, los documentos extensos siempre suben en el ranking.

BM25 es la fórmula que la comunidad de IR adoptó para corregir ambos problemas. Hoy es el default en Elasticsearch, OpenSearch, Lucene y Tantivy.

¿De dónde sale el nombre "BM25"?

Best Matching, variante **25** del programa de investigación de **Stephen Robertson y Karen Spärck Jones** en City University London (1976–1994). Literalmente: probaron 24 fórmulas antes; la número 25 fue la que funcionó.

Salió de la **teoría probabilística** del ranking: el score "correcto" es el log-odds de que un documento sea relevante dada la consulta.

Intuición geométrica: **TF-IDF con dos manijas** — una para saturar el TF (k_1), otra para penalizar el largo (b). Los valores estándar $k_1 \approx 1.2$, $b \approx 0.75$ salieron de décadas de experimentos empíricos.

La fórmula

$$\text{BM25}(t, d) = \underbrace{\text{IDF}(t)}_{\text{rareza global}} \cdot \underbrace{\frac{\text{TF}(t, d) \cdot (k_1 + 1)}{\text{TF}(t, d) + k_1 \cdot L(d)}}_{\text{TF saturado y normalizado por largo}}$$

con $L(d) = 1 - b + b \cdot \frac{|d|}{\text{avgdl}}$ (el factor de largo).

- Doc de largo **promedio**: $L(d) = 1 \rightarrow$ "TF-IDF con saturación".
- Doc **más largo** que el promedio: $L(d) > 1 \rightarrow$ más castigo.
- Doc **más corto**: $L(d) < 1 \rightarrow$ score más alto.

Cómo se ve la saturación

Contribución del factor TF de BM25 (con $k_1 = 1.2$):

TF en el doc	TF-IDF (lineal)	BM25 (factor TF)
1	1	1.00
5	5	1.83
20	20	2.07
100	100	2.18
1.000	1.000	2.20

→ BM25 se acerca asintóticamente a $k_1 + 1 = 2.2$. Esto pone un límite en TF, que aunque usemos logaritmo crecería infinitamente.

Cómo se ve la normalización por largo

Con $b = 0.75$ y un término que aparece 5 veces:

Largo del doc / promedio	Factor en el denominador	Score relativo
0.5× (párrafo corto)	0.625	más alto
1.0× (largo promedio)	1.000	base
2.0× (doble)	1.375	más bajo
10× (libro entero)	8.125	muy bajo

→ Un libro que menciona "gato" 50 veces puede rankear **por debajo** de un párrafo que lo menciona 5.

De sparse a dense

BM25 es cosine similarity sobre vectores sparse

Desde la geometría: BM25 es **cosine similarity** entre consulta y documento, en un espacio donde **cada dimensión es un término**.

```
vocab = [base, dato, gato, ojo, mundo, ...]    (~100.000 dims, una por stem)
d1 ("ojo por ojo")      → [0, 0, 0, 4.32, 0, ...]
q  ("ojo del huracán") → [0, 0, 0, 2.20, 0, ...]
```

$$\text{score}(q, d) = \cos(q, d) = \frac{q \cdot d}{\|q\| \cdot \|d\|}$$

- Casi todas las entradas son 0 → vector **sparse**.
- BM25 (o TF-IDF) = **esquema de pesos** para las entradas no-cero.
- El **índice invertido** hace esta cosine tratable: solo guarda las entradas no-cero, agrupadas por término.

Aprender los pesos en vez de calcularlos

¿Y si los pesos no vienen de la fórmula de Robertson, sino de un **modelo entrenado**?

Modelos como **SPLADE** (Stanford, 2021) y variantes:

- Un transformer toma el documento y produce un vector **del mismo formato sparse** — una dimensión por término del vocabulario.
- Los pesos no son TF·IDF·corrección, sino la salida de una **red entrenada** con datos de búsqueda real.
- Como el formato no cambia, **se indexa con el mismo índice invertido** y se consulta igual.

→ "BM25 con esteroides ML": mismo plumbing, mejor calidad de ranking.

Otro camino: vectores densos

Las representaciones internas de un transformer **ya son vectores**. En vez de una dimensión por término, el modelo colapsa el documento en un vector **denso** de ~768 dimensiones, todas distintas de cero:

```
d1 ("ojo por ojo")      → [ 0.12, -0.48, 0.81, ...]   (768 dims)
d2 ("ojo iracundo")    → [ 0.11, -0.45, 0.79, ...]   (cerca de d1)
d3 ("mundo cruel")    → [-0.22, 0.17, -0.34, ...]   (lejos)
```

- Embebes **consulta y documento con el mismo modelo** (BERT, sentence-transformers, OpenAI embeddings, ...).
- **Misma operación de antes:** cosine similarity, ahora en un espacio denso de pocas dimensiones.
- Captura **semántica:** `coche` y `automóvil` quedan cerca aunque no compartan letras.

Y en PostgreSQL: **pgvector**

```
CREATE EXTENSION vector;  
ALTER TABLE articulos ADD COLUMN embedding vector(768);  
CREATE INDEX ON articulos USING hnsw (embedding vector_cosine_ops);  
  
SELECT id, titulo FROM articulos  
ORDER BY embedding <=> $1      -- cosine distance  
LIMIT 10;
```

- Léxico y semántico **conviven en el mismo PostgreSQL**.
- Hybrid search en PG = combinar @@ (sparse) + <=> (dense).

Mas detalles: **Procesamiento de Datos Masivos**.

Más allá del curso

Las ideas de hoy son la base de:

- **Elasticsearch / OpenSearch / Lucene / Tantivy**: índice invertido + BM25 a gran escala, distribuido. Motor de búsqueda interna de prácticamente cualquier aplicación grande.
- **RAG (Retrieval-Augmented Generation)**: hybrid retrieval → LLM. La búsqueda léxica es el cimiento del retrieval moderno, **no quedó obsoleta** cuando aparecieron los embeddings — quedó **complementada**.

Lo aprendido hoy es el cimiento sobre el que se construye la búsqueda en internet, en tu correo, y la mitad del stack de IA actual.

Resumen

- En PostgreSQL: `to_tsvector` + `to_tsquery` + `@@` + índice **USING GIN** = pipeline + índice invertido + matching.
- `ts_rank` da ranking básico pero **no es TF-IDF completo** (sin IDF global).
- **BM25** = TF-IDF + saturación de TF + normalización por largo. Default de producción.
- BM25/TF-IDF producen vectores **sparse**; embeddings producen vectores **dense**.
Hybrid combina ambos — es la base del RAG moderno.

Próxima semana

- Cambiamos de tema: **Data Warehousing**.
- Volvemos a SQL — pero ahora para **análisis** (OLAP) sobre datos históricos consolidados.
- Re-visitamos la normalización de Semana 5 y la **invertimos deliberadamente** (¡desnormalizar!). El por qué tiene que ver, otra vez, con costos de I/O.